TP 11 : Synchronisation Locale / Cloud
Firebase RTDB + SharedPreferences

Développement Mobile Android

Contexte

Vous allez développer une application Android nommée :

SmartNotes

SmartNotes est une application permettant :
— d'ajouter des notes personnelles (Titre, Description, Date)
— de choisir un mode d'affichage (Dark Mode)
— de synchroniser les notes avec Firebase Realtime Database
— de sauvegarder automatiquement l'état local du formulaire
Ce TP a pour objectif de relier :
— Firebase RTDB (stockage cloud)
— SharedPreferences (stockage local léger)

Objectifs pédagogiques

À la fin de ce TP, vous devez être capables de :
— Utiliser DatePickerDialog
— Sauvegarder des données simples avec SharedPreferences
— Restaurer l'état d'un formulaire complet
— Gérer une première ouverture d'application
— Sauvegarder des préférences utilisateur (Dark Mode)
— Synchroniser des données vers Firebase RTDB
— Comprendre la différence entre cache Firebase et SharedPreferences

Partie 1 : Création du formulaire complet

Créer une activité contenant :
— EditText : Titre
— EditText : Description
— EditText : Date (ouvre automatiquement un calendrier)
— Switch : Mode sombre
— Button : Enregistrer

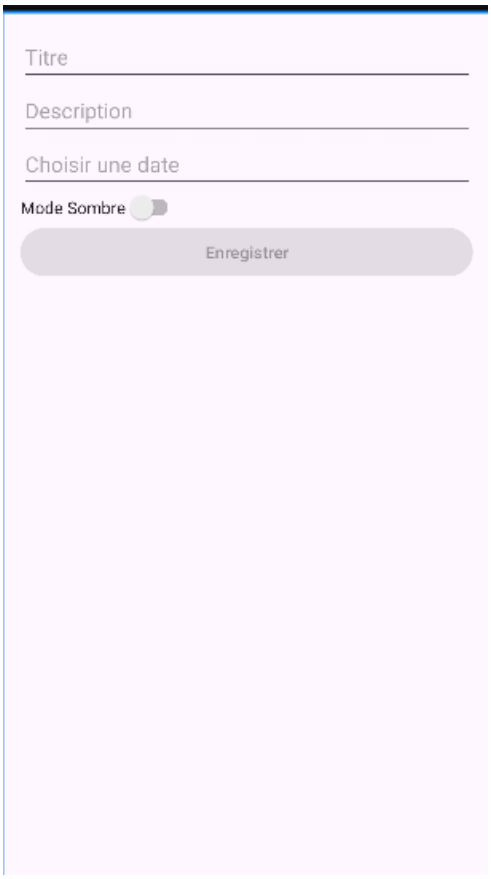

FIGURE 1 – Interface d'application

Champ Date

Le champ Date doit ouvrir un calendrier Android via **DatePickerDialog**. La date sélectionnée doit s'afficher dans l'EditText.

La date sera stockée sous forme de String.

Validation

Le bouton **Enregistrer** ne doit être activé que si :

— Titre non vide
— Description non vide
— Date sélectionnée

Partie 2 : Sauvegarde automatique du formulaire

Objectif : Ne pas perdre les données si l'application est fermée avant l'envoi.

Travail demandé

Sauvegarder dans SharedPreferences :

— Titre
— Description
— Date

— Etat du Switch Dark Mode

Dans MainActivity.java, dans la méthode onCreate, on ouvre un fichier SharedPreferences nommé "SmartPrefs" en mode privé pour stocker des données locales de l'application. :

```
SharedPreferences prefs =
getSharedPreferences("SmartPrefs", MODE_PRIVATE);
```

On vérifie par la suite, si l'option Dark Mode a été sauvegardée dans SharedPreferences.

Si la valeur est true, l'application active le thème sombre grâce à AppCompatDelegate.

```
// DARK MODE
boolean darkMode = prefs.getBoolean("darkMode", false);
if (darkMode) {
    AppCompatDelegate.setDefaultNightMode(AppCompatDelegate.
       MODE_NIGHT_YES);
}
```

Après, il faut passer à la récupération des views de l'interface activity_main.xml, et à récupérer la référence de la base de données Firebase vers le nœud "notes", afin d'y lire ou enregistrer des données.

```
editTitre = findViewById(R.id.editTitre);
editDescription = findViewById(R.id.editDescription);
editDate = findViewById(R.id.editDate);
switchDark = findViewById(R.id.switchDark);
btnSave = findViewById(R.id.btnSave);

databaseReference = FirebaseDatabase.getInstance().getReference("
   notes");

restoreData();
checkFirstTime();
setupDatePicker();
setupValidation();
setupDarkMode();
setupSaveButton();
```

Explication des fonctions principales :

- **restoreData()** : Restaure les données sauvegardées dans `SharedPreferences` et les affiche dans les champs du formulaire.
- **checkFirstTime()** : Vérifie si c'est le premier lancement de l'application et affiche un message de bienvenue si nécessaire.
- **setupDatePicker()** : Configure l'ouverture du calendrier (`DatePickerDialog`) lors du clic sur le champ date.
- **setupValidation()** : Met en place la validation dynamique afin d'activer le bouton uniquement si tous les champs sont remplis.
- **setupDarkMode()** : Gère l'activation et la sauvegarde du mode sombre selon l'état du `Switch`.
- **setupSaveButton()** : Définit l'action du bouton `Enregistrer` pour envoyer les données vers Firebase et vider le formulaire.

DatePicker

La méthode **setupDatePicker()** ouvre un `DatePickerDialog` lorsque l'utilisateur clique sur le champ Date. Elle récupère la date actuelle puis affiche le calendrier pour permettre la sélection d'une nouvelle date. La date choisie est affichée dans l'EditText et sauvegardée automatiquement avec `saveFormData()`.

```
private void setupDatePicker() {
        editDate.setOnClickListener(v -> {

            Calendar calendar = Calendar.getInstance();
            int year = calendar.get(Calendar.YEAR);
            int month = calendar.get(Calendar.MONTH);
            int day = calendar.get(Calendar.DAY_OF_MONTH);

            DatePickerDialog datePicker = new DatePickerDialog(
                    this,
                    (view, selectedYear, selectedMonth, selectedDay
                       ) -> {

                        String date = selectedDay + "/" +
                                (selectedMonth + 1) + "/" +
                                selectedYear;

                        editDate.setText(date);
                        saveFormData();

                    }, year, month, day);

            datePicker.show();

        });
    }
```

setupValidation

Cette méthode met en place un `TextWatcher` pour écouter les modifications du texte dans les champs Titre et Description. À chaque changement, elle appelle `validateFields()` pour vérifier si tous les champs sont remplis afin d'activer ou désactiver le bouton, puis elle appelle `saveFormData()` pour sauvegarder automatiquement les données dans `SharedPreferences`.

```
private void setupValidation() {

        TextWatcher watcher = new TextWatcher() {
            @Override public void beforeTextChanged(CharSequence s,
                int start, int count, int after) {}
            @Override public void onTextChanged(CharSequence s, int
                start, int before, int count) {
                validateFields();
                saveFormData();
            }
```

```
            @Override public void afterTextChanged(Editable s) {}
        };

        editTitre.addTextChangedListener(watcher);
        editDescription.addTextChangedListener(watcher);
    }
```

validateFields

Créer une méthode `validateFields()` qui vérifie que les champs Titre, Description et Date ne sont pas vides en utilisant `toString().isEmpty()`. Déclarer une variable booléenne `valid` qui sera vraie uniquement si les trois champs sont remplis. Activer ou désactiver le bouton `btnSave` en fonction de la valeur de cette variable.

saveFormData

Cette méthode enregistre automatiquement les valeurs des champs Titre, Description et Date dans `SharedPreferences` à l'aide d'un `Editor`. Elle utilise `apply()` pour sauvegarder les données de manière asynchrone afin de conserver l'état du formulaire.

```
private void saveFormData() {
    SharedPreferences.Editor editor = prefs.edit();
    editor.putString("titre", editTitre.getText().toString());
    editor.putString("description", editDescription.getText().
       toString());
    editor.putString("date", editDate.getText().toString());
    editor.apply();
    }
```

Restauration

Dans `onCreate()`, restaurer toutes les données :

```
private void restoreData() {
    editTitre.setText(prefs.getString("titre", ""));
    editDescription.setText(prefs.getString("description", ""));
    editDate.setText(prefs.getString("date", ""));
    switchDark.setChecked(prefs.getBoolean("darkMode", false));
    validateFields();
    }
```

Tester :

- Fermer l'application sans enregistrer
- La rouvrir
- Vérifier que tout le formulaire est restauré

clearForm

Cette méthode vide les champs du formulaire en réinitialisant leur contenu à une chaîne vide. Elle supprime également les données correspondantes dans `SharedPreferences`, puis appelle `validateFields()` pour mettre à jour l'état du bouton.

```java
private void clearForm() {
    editTitre.setText("");
    editDescription.setText("");
    editDate.setText("");

    SharedPreferences.Editor editor = prefs.edit();
    editor.remove("titre");
    editor.remove("description");
    editor.remove("date");
    editor.apply();

    validateFields();
    }
```

setupDarkMode

Cette méthode configure un `OnCheckedChangeListener` sur le `Switch` pour détecter l'activation ou la désactivation du mode sombre. Elle sauvegarde le choix dans `SharedPreferences` puis applique immédiatement le thème sombre ou clair grâce à `AppCompatDelegate`.

```java
private void setupDarkMode() {
    switchDark.setOnCheckedChangeListener((buttonView, isChecked)
       -> {

        SharedPreferences.Editor editor = prefs.edit();
        editor.putBoolean("darkMode", isChecked);
        editor.apply();

        if (isChecked) {
            AppCompatDelegate.setDefaultNightMode(AppCompatDelegate
               .MODE_NIGHT_YES);
        } else {
            AppCompatDelegate.setDefaultNightMode(AppCompatDelegate
               .MODE_NIGHT_NO);
        }
    });
    }
```

Partie 3 : Synchronisation Firebase

Créer une classe Note contenant :
- String titre
- String description
- String date

Lorsque l'utilisateur clique sur **Enregistrer** :
- Envoyer l'objet Note vers Firebase RTDB
- Supprimer les données sauvegardées dans SharedPreferences

```
Note note = new Note(titre, description, date);
databaseReference.push().setValue(note);

editor.clear();
editor.apply();
```

Créer une méthode `setupSaveButton()` qui associe un `OnClickListener` au bouton `btnSave`. À l'intérieur du clic, récupérer les valeurs des champs, créer un objet `Note` puis l'envoyer vers Firebase en utilisant `push().setValue()`. Afficher un message de confirmation avec `Toast` et appeler une méthode `clearForm()` pour vider le formulaire.

Partie 4 : Première ouverture

Ajouter un écran d'introduction affiché uniquement au premier lancement.

```
boolean firstTime = prefs.getBoolean("firstTime", true);
```

Si true :
- Afficher un message de bienvenue
- Sauvegarder firstTime = false

```
private void checkFirstTime() {
        boolean firstTime = prefs.getBoolean("firstTime", true);

        if (firstTime) {
            Toast.makeText(this, "Bienvenue␣dans␣SmartNotes␣!",
               Toast.LENGTH_LONG).show();

            SharedPreferences.Editor editor = prefs.edit();
            editor.putBoolean("firstTime", false);
            editor.apply();
        }
    }
```

Partie 5 : Comparaison théorique

Critère	SharedPreferences	Firebase RTDB
Type de données	Données simples (String, int, boolean, float, long) sous forme clé/valeur	Données structurées en JSON (objets, listes, arborescence)
Stockage local	Oui (mémoire interne de l'application)	Non (stockage principal dans le cloud)
Synchronisation automatique	Non (gestion manuelle par le développeur)	Oui (mise à jour temps réel entre appareils)
Multi-appareil	Non (données limitées à un seul appareil)	Oui (données partagées entre plusieurs appareils)

Sauvegarde formulaire non envoyé	Oui (peut sauvegarder les champs avant envoi)	Non (ne sauvegarde que les données envoyées au serveur)
Gestion de l'état UI	Oui (thème, première ouverture, champs en cours)	Non (ne gère pas l'état de l'interface utilisateur)

Questions de réflexion

1. Pourquoi Firebase persistence ne permet-elle pas de restaurer un formulaire non envoyé ?
2. Pourquoi la date est-elle stockée sous forme de String ?
3. Quel format de date permettrait un tri correct (dd/MM/yyyy ou yyyy-MM-dd) ?
4. Dans quel cas utiliser SQLite au lieu de SharedPreferences ?
5. Quelle solution choisir pour partager des données entre plusieurs téléphones ?

Conclusion

Dans ce TP vous avez combiné :
— Stockage local léger (SharedPreferences)
— Synchronisation Cloud (Firebase RTDB)
— Gestion d'état de l'application
— Gestion d'une donnée de type Date
Vous avez compris que :

SharedPreferences = Mémoire locale de l'application
Firebase = Mémoire partagée dans le Cloud